This is a postprint version of the following published document:

Blanco-Romero, J., Lorenzo, V., Almenares, F., Sánchez, D. D., Campo, C., & Rubio, C. G. (2024). Integrating Post-Quantum Cryptography into CoAP and MQTT-SN Protocols. In *2024 IEEE Symposium on Computers and Communications (ISCC)* (pp. 1–6).

DOI: 10.1109/iscc61673.2024.10733716

© IEEE Xplore. Personal use of this material is permitted. Permission from IEEE must be obtained for all other uses, in any current or future media, including reprinting/republishing this material for advertising or promotional purposes, creating new collective works, for resaleor redistribution to servers or lists, or reuse of any copyrighted component of this work in other works.

Integrating Post–Quantum Cryptography into CoAP and MQTT–SN Protocols

Javier Blanco–Romero
Departamento de Ingeniería Telemática
Universidad Carlos III de Madrid
Madrid, Spain
frblanco@pa.uc3m.es

Vicente Lorenzo
Departamento de Ingeniería Telemática
Universidad Carlos III de Madrid
Madrid, Spain
vilorenz@pa.uc3m.es

Florina Almenares
Departamento de Ingeniería Telemática
Universidad Carlos III de Madrid
Madrid, Spain
florina@it.uc3m.es

Daniel Díaz Sánchez
Departamento de Ingeniería Telemática
Universidad Carlos III de Madrid
Madrid, Spain
dds@it.uc3m.es

Celeste Campo
Departamento de Ingeniería Telemática
Universidad Carlos III de Madrid
Madrid, Spain
celeste@it.uc3m.es

Carlos García Rubio
Departamento de Ingeniería Telemática
Universidad Carlos III de Madrid
Madrid, Spain
cgr@it.uc3m.es

***Abstract*—Post–Quantum Cryptography (PQC) is a practical and cost–effective solution to defend against emerging quantum computing threats. So, leading worldwide security agencies and standardization bodies strongly advocate for the proactive integration of PQ cryptography into underlying frameworks to support applications, protocols, and services. The current research predominantly addresses the incorporation of PQC in Internet communication protocols such as HTTP and DNS; nevertheless, the focus on embedded devices has been limited to evaluating PQC's integration within TLS/DTLS in isolation. Hence, there is a notable gap in understanding how PQC impacts IoT–specific communication protocols. This paper presents the integration of PQC into two communication protocols specifically tailored for IoT devices, the Constrained Application Protocol (CoAP) and MQTT for Sensor Networks (MQTT–SN), via the wolfSSL library. These two integrations contribute to the understanding of PQC's implications for IoT communication protocols.**

***Index Terms*—Post–Quantum Cryptography, PQC, IoT protocols, CoAP, MQTT–SN**

I. Introduction

Building on the pioneering cryptanalysis of the mathematical foundations of traditional cryptography by Shor and Grover [1], [2], advancements in quantum computing are estimated to pose a threat to current cryptographic methods within the next 6 to 20 years [3]. This timeline suggests that without adaptation, traditional asymmetric algorithms will become ineffective and vulnerable against quantum computing. To overcome this threat the usage of Post–Quantum Cryptography (PQC) is being proposed [4] and standardised [5]–[7].

Leading worldwide cybersecurity agencies and standardization bodies, such as NIST (National Institute of Standards and Technology) [8], the UK NCSC (National Cyber Security Centre) [9], NSA (National Security Agency) [10], CISA (Cybersecurity and Infrastructure Security Agency), ENISA (European Union Agency for Cybersecurity), the Spanish CCN ("Centro Criptológico Nacional") [11], the French ANSSI ("*Agence Nationale de la Sécurité des Systèmes d'Information*"), the German Federal Office for Information Security (BSI, "*Bundesamt für Sicherheit in der Informationstechnik*"), NLNCSA (Netherlands National Communications Security Agency), and the Swedish NCS (National Cyber Security Centre) [12], have collectively emphasized the critical need to transition towards PQC to defend against the emerging quantum computing threat and for comprehensive protection [8]. They underscore the necessity of migrating to and integrating PQC into existing cryptographic frameworks to ensure a seamless transition to quantum–resistant technologies [12], due to its readiness and critical role in maintaining secure cryptographic standards.

In this way, several research works are being carried out on the integration of post–quantum cryptography into secure communication protocols such as (D)TLS ((Datragram) Transport Layer Security) [13], IPSec (Internet Protocol Security) [14], [15], and DNSSEC (Domain Name System Security Extensions) [16]. Likewise, a new IETF (Internet Engineering Task Force) working group has been created, Post–Quantum Use In Protocols (PQUIC). Nevertheless, the integration of PQC on protocols that form part of IoT (Internet of Things) or IIoT (Industrial IoT) scenarios is absent or still in the early stages. This involves some challenges, because PQ primitives often offer larger signatures or keys –than traditional cryptography– or require, in some cases, more computational capabilities [17]. Other challenges include the lack of standardized methods to measure the security levels of PQC against quantum attacks, the need for energy–efficient PQC algorithms that do not compromise security over time, and the optimization of specific computational tasks in PQC algorithms for IoT microcontrollers and network architectures [18]. However, it is important to overcome such challenges and perform the PQ integration, because such IoT devices may run critical applications or handle sensitive data in healthcare systems, automation systems, connected vehicles, or smart grids environments, among others.

This paper presents novel contributions to the integration of PQC in (I)IoT communication protocols, addressing the current research gap. We focus on CoAP (Constrained Application Protocol) and MQTT-SN (Message Queuing Telemetry Transport for Sensor Networks), integrating the wolfSSL library, which incorporates PQC into DTLS 1.3, into the libcoap and Paho MQTT-SN Gateway implementations. Our work aims to study the viability and performance of PQC in applications running such protocols, contribute to the open-source development of these integrations to enhance crypto-agility, and identify potential obstacles and lessons learned from a software perspective.

The remainder of this paper is organized as follows. Section II introduces briefly CoAP and MQTT–SN, highlighting the necessity of using such lightweight protocols when dealing with IoT devices. In Section III an overview of the existing literature is provided, presenting key insights and contributions in the field to contextualize the current study. Section IV describes the integration performed in CoAP and MQTT–SN libraries. Finally, Section VI is devoted to summarize the key findings and implications of the present paper outlining avenues for further research in the subject area.

II. Background

A. IoT Protocols

The Constrained Application Protocol [19] and MQTT for Sensor Networks [20] are two widely adopted protocols in the IoT domain. CoAP is a request–response protocol that operates mainly over UDP and is suitable for RESTful architectures. MQTT–SN is a publish–subscribe protocol designed for constrained devices and low–bandwidth networks. MQTT–SN architecture consists of clients, gateways (GWs), and forwarders. Clients communicate with an MQTT server through a GW, while the GW translates between MQTT and MQTT–SN protocols. Forwarders can be used when the GW is not directly accessible. MQTT–SN gateways can be classified as transparent or aggregating based on their protocol translation approach. Transparent gateways establish separate MQTT connections for each client, while aggregating gateways maintain a single MQTT connection and handle message exchanges locally. As both CoAP and MQTT–SN typically run over UDP, their communications can be secured using DTLS, a UDP–based variant of TLS. DTLS provides secure communications, but is not optimized for constrained environments. DTLS consists of a handshake phase for authentication and key agreement, and a data transmission phase for sending encrypted data using the established keys.

B. IoT Protocol Implementations

The libcoap library [21] and Eclipse Paho MQTT–SN [22] are reference implementations of CoAP and MQTT–SN, respectively. The integration of PQC into CoAP and MQTT–SN, described in the present paper (see Section IV below), relies on these libraries.

The libcoap C library was developed as part of an open–source initiative and addresses the need for efficient communication in environments limited by computing power, memory, bandwidth, and network packet sizes. It does not support DTLS over multicast currently [23].

The Eclipse Paho MQTT–SN project has been developed in C/C++ for embedded platforms. It comprises three sub–projects: `MQTTSNPacket`, a low–level library for MQTT–SN packet serialization and deserialization; `MQTTGateway`, an MQTT–SN transparent/aggregating gateway that connects MQTT–SN clients with an MQTT server; and `MQTTSNClient`, a higher–level C++ client library (currently incomplete). The `MQTTGateway` component, which facilitates communication between MQTT–SN and MQTT, can operate as either a transparent gateway or an aggregating gateway, depending on the configuration settings, aligning with the MQTT–SN specification. The `MQTTGateway` supports diverse transports for IoT connectivity, including secure DTLS/DTLS6, long–range LoRaLink, Bluetooth RFCOMM, standard UDP/UDP6, and XBee for mesh networks.

III. Related Work

This section presents previous and related work in security and PQC for IoT.

A. Protocols and Security in Constrained IoT Environments

Several studies have explored the impact of security mechanisms on various protocols in constrained environments. For example, Silva et al. [24] found that security algorithms significantly impact message size and execution times in WSNs (Wireless Sensor Networks) using HTTP and MQTT, with faster CPUs consuming less power. Seoane et al. [25] analyzed CoAP and MQTT performance, revealing that adding security significantly increased bandwidth usage (over 1000% in CoAP and 74–200% in MQTT) and CPU usage (3.5–11.5% in CoAP and 27–36% in MQTT). Kwon et al. [26] investigated the performance of `RawPublicKey` and `PreSharedKey` modes in the CoAP/DTLS binding, concluding that while Class 1 devices can effectively use `PreSharedKey`, the mandatory `RawPublicKey` mode cannot be efficiently operated on Class 1 and Class 2 devices due to the higher computational overhead of asymmetric–key cryptography.

B. Post–Quantum Security in Constrained Environments

The Open Quantum Safe project [27] incorporates liboqs library [28], through the Provider API using the `oqs-provider` [29]. liboqs implements and adapts post–quantum cryptographic algorithms from open–source projects like PQClean [30] and also has own implementations. It integrates with well-known crypto libraries such as OpenSSL, OpenSSH, and the embedded–focused wolfSSL [31], which supports PQ KEMs (Key Encapsulation Mechanisms), hybrid KEMs, and signatures with DTLS 1.3. Another embedded–optimized library is pqm4 [32], a collection of PQC algorithms for ARM Cortex–M4.

Early studies explored the NTRU lattice–based cryptosystem for IoT devices and DTLS, showcasing its feasibility, efficiency, and reduced memory footprint [33], [34], although

NTRU cryptosystem is not being considered for standardization by NIST [35]. Burstinghaus et al. [18] integrated the Kyber key encapsulation mechanism and SPHINCS+ signature schemes into the mbedTLS library, replacing Diffie–Hellman and ECDSA (Elliptic Curve Digital Signature Algorithm) in the TLS 1.2 handshake. While Kyber outperformed Elliptic Curve DH on various platforms, SPHINCS+ signatures were generally slower than ECDSA due to their larger size and signing time. Gonzalez et al. [36] proposed streaming public keys and signatures over the in–vehicle network for PQC signature verification on severely resource–constrained ARM Cortex–M3 devices, efficiently using limited memory. Gonzalez and Wiggers [37] investigated KEMTLS, an alternative TLS handshake protocol that authenticates peers through long–term KEM keys in certificates, showing reduced handshake time, lower peak memory consumption, and saved traffic volume compared to TLS 1.3 in IoT network settings. Tasopoulos et al. [38], [39] conducted comprehensive evaluations of post–quantum TLS 1.3 on constrained embedded devices, assessing execution time, memory usage, bandwidth requirements, and energy consumption. Their findings revealed that the choice of PQC algorithms significantly impacts energy efficiency. Septien–Hernandez et al. [40] evaluated the performance and energy consumption impacts of post–quantum KEMs on resource–constrained IoT devices using an MQTT architecture, identifying Kyber512 and LightSaber as the best–performing KEMs, with a trade–off between energy consumption and memory storage. Finally, McLoughlin et al. [41] evaluated a post–quantum DTLS 1.3 implementation using the Dilithium signature scheme and Kyber KEM on an IoT device, finding increased communication and memory costs but significantly lower computational costs compared to traditional DTLS 1.3. They concluded that a PQ DTLS 1.3 version using Dilithium and Kyber is feasible for IoT, with future research exploring more resource–friendly PQ algorithms or the effects of compression on communication costs.

C. Integration of PQC in (D)TLS

Open source efforts to integrate PQC into the widely used OpenSSL library are evident in the Open Quantum Safe project [27].This incorporates PQ algorithms in TLS 1.3, being possible to use a hybrid solution as proposed in [13]. However, it is important to note that these implementations are not optimized for embedded systems [18].

By the other hand, wolfSSL, a lightweight and open–source cryptographic library designed for embedded systems, provides supports post–quantum KEMs, hybrid KEMs and post–quantum signatures with DTLS 1.3 [31].

To integrate PQC into CoAP and MQTT–SN, we will extend both libcoap and the Paho MQTT–SN Gateway to support the use of the wolfSSL library for transport.

IV. Integrating PQC into CoAP and MQTT–SN

The integration of PQC in CoAP and MQTT–SN libraries leverages the existing wolfSSL PQC integration and the OpenSSL Compatibility Layer [42]. Both libcoap and Paho implementations support natively the use of OpenSSL as the underlying DTLS library. The compatibility layer allows an easy integration of PQC into these protocols by allowing a straightforward transition from OpenSSL to wolfSSL. This strategy aims to minimize the need for extensive code modifications when migrating to wolfSSL. The compatibility layer's capability to bridge the gap between OpenSSL and wolfSSL APIs streamlines the process of incorporating PQ cryptographic algorithms into the existing CoAP and MQTT–SN gateway codebases.

In migrating to wolfSSL, it is important to address differences, especially in DTLS communication and handshake management. OpenSSL uses `DTLSv1_listen()` for the initial handshake via a listening UDP socket, this is necessary for a stateless handshake with `HelloVerifyRequest`, mitigating DoS attacks by avoiding premature resource allocation. In contrast, wolfSSL does not have a direct equivalent of `DTLSv1_listen()`, managing stateless handshakes through `wolfSSL_accept()` state machine, showcasing a distinct approach compared to OpenSSL.

A. CoAP

Our wolfSSL integration into libcoap is open–sourced in a fork [43]. Taking ideas from our work, a recent pull request [44] integrating wolfSSL has been merged into the libcoap development branch.

The DTLS integration in libcoap is modularized across different `coap_*.c` source files for various TLS libraries like openssl, tinydtls, mbedtls, gnutls, and notls. This modular design facilitates creating a new `coap_wolfssl.c` file based on `coap_openssl.c` to integrate wolfSSL using the compatibility layer.

The interaction of the CoAP example server and client with the main functions in the `coap_openssl.c` to perform the basic (D)TLS operations, such as executing the handshake to establish a session and writing/reading encrypted application data, is depicted in Figure 1.

B. MQTT–SN

We have integrated wolfSSL into both the Paho MQTT–SN Gateway and the GatewayTester publisher and subscriber clients 2 to enable secure communication with PQC. in MQTT–SN. Our fork of the Paho MQTT–SN Gateway with wolfSSL integration for PQC is available at [45]. The following section detail the integration process for the Gateway. The integration into the GatewayTester is parallel and will not be further discussed.

For client communication, the Gateway acts as a DTLS server, with the core DTLS interface in `SensorNetwork.cpp`. `SensorNetwork::read()` handles DTLS connections and application data reception, we implemented a custom `BIO` (Basic Input/Output) object for UDP sockets using `wolfSSL_BIO_meth_new()` since wolfSSL lacks an equivalent to OpenSSL's `BIO_new_dgram()`. In the original MQTT–SN Gateway

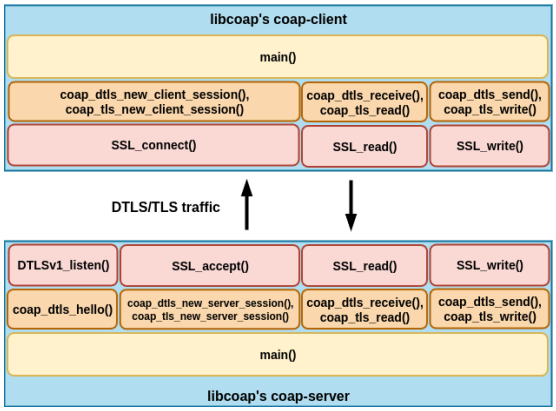

Fig. 1. Architecture of the libcoap server and client, illustrating the interaction between libcoap functions and the underlying OpenSSL's `SSL_*` functions. The `coap_openssl.c` file implements the OpenSSL DTLS transport in the functions depicted by the orange boxes, which contain calls to the OpenSSL's `SSL_*` functions (in red) responsible for the handshake and encrypted read/write operations. The yellow boxes represent the main loops in `examples/coap-server.c` and `examples/coap-client.c`.

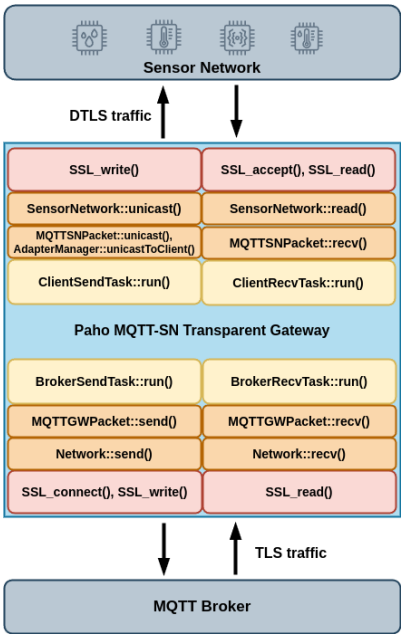

Fig. 2. Architecture of the Paho MQTT-SN Gateway, depicting the DTLS and TLS communication with sensor clients and the MQTT broker, respectively. The diagram illustrates the function call stack from the `Send` and `Recv` loops (yellow) for clients and the broker, passing through the packet and network layers (orange). The network layer uses OpenSSL's `SSL_*` functions to perform the handshake and encrypted communication.

implementation, `DTLSv1_listen()` is used to process initial `ClientHello` messages from clients during the DTLS handshake. If successful, it creates a dedicated socket `client_fd` for further communication, bound to the client's address. This allows completing the handshake via `SSL_accept()` and attaching the client to a specific session. However, since wolfSSL lacks a direct equivalent to `DTLSv1_listen()`, we use the wolfSSL callback function `wolfDTLS_SetChGoodCb()` to handle the client socket setup. This callback is invoked after verifying the `ClientHello`. Inside this callback, a dedicated client socket is created, bound to the server, and connected to the client. The wolfSSL `BIO` is then updated to use this client socket and set the peer/connected state via `wolfSSL_BIO_ctrl()`. Once the DTLS connection is established, `SensorNetwork::read()` iterates through connections, using `wolfSSL_read()` to decrypt and retrieve MQTT–SN application data. For client message dispatch, the Gateway uses `SensorNetwork::unicast()` with `SSL_write()` to transmit MQTT–SN messages over DTLS, orchestrated through functions like `MQTTSNPacket::unicast()`, `AdapterManager::unicastToClient()`, and `ClientSendTask::run()`. When communicating with the MQTT Broker, the Gateway operates as a TLS client (`Network.cpp`). Connections use `Network::connect()` with `SSL_connect()`. Message transmission uses `Network::send()` with `SSL_write()`, and reception uses `Network::recv()` with `SSL_read()`.

V. Preliminary PQC–based CoAP and MQTT–SN Performance

We have deployed a set of preliminary tests to evaluate the libcoap and Paho MQTT–SN Gateway integrations and performance to gain initial insights. In particular, we intend to verify whether relying on post–quantum or hybrid cryptography falls behind relying on classic cryptography. In both cases, the following setups have been considered:

1) wolfSSL's implementation of DTLS v1.3 with the classic P256 algorithm.
2) wolfSSL's implementation of DTLS v1.3 with the post–quantum Kyber512 algorithm.
3) wolfSSL's implementation of DTLS v1.3 with the hybrid P256–Kyber512 algorithm.

For libcoap, the tests involved running a libcoap server and sequentially sending a single GET request from 100 clients. Each experiment was monitored using Wireshark to measure the time required for the server to complete the GET requests. As shown in Figure 4, introducing post–quantum and hybrid algorithms does not appear to have a significant detrimental effect on the metric. Specifically, the figure depicts the time in seconds per client's request, with Kyber512 exhibiting the best performance at 0.08 seconds. The traditional ECC–based P256 scheme took approximately 0.10 seconds, while the hybrid Kyber512–P256 scheme required nearly 0.11 seconds.

For the MQTT–SN experiments, we used the Paho MQTT–SN Gateway running on the Raspberri Pi as an aggregating gateway and the GatewayTester subscriber client. We measured the time of the entire subscription process for 100 sequential subscribers using the same algorithms as in the CoAP experiments. The results are shown in Figure 3. It is worth noting that the subscription time in the MQTT–SN experiments is roughly one order of magnitude greater than the pure DTLS handshake time. Consequently, the time impact of

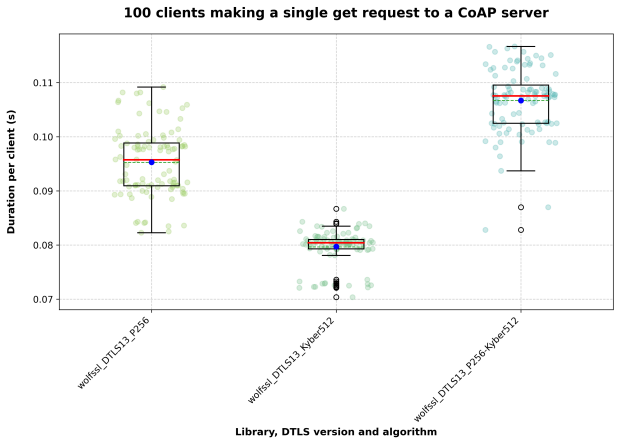

Fig. 3. Preliminary libcoap benchmark.

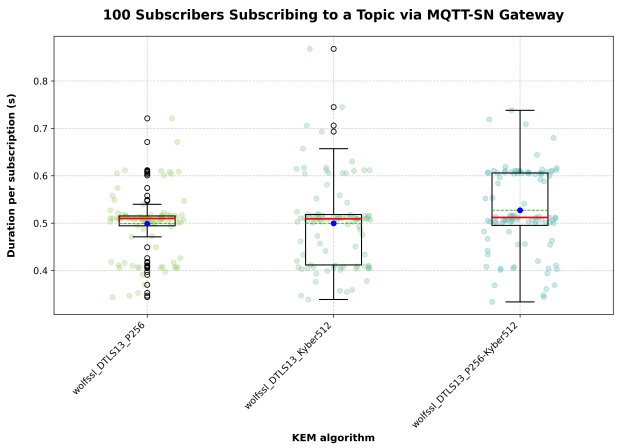

Fig. 4. Preliminary MQTT–SN Gateway benchmark.

PQC on the entire MQTT–SN subscription process is close to negligible.

The experiments were conducted using two systems: a Raspberry Pi 4 Computer Model B with a 64–bit quad–core Cortex–A72 processor, 4 GB RAM, and DietPi v8.20 operating system, serving as the server/gateway; and a PC with an Intel Core i7–7700 CPU (4 cores, 8 threads, 3.6 GHz base clock, up to 4.2 GHz turbo frequency), 32 GB RAM, and Ubuntu 22.04 operating system, acting as the client/subscriber.

VI. Conclusions and Future Work

This paper presented two implementations integrating PQC algorithms into the IoT communication protocols CoAP and MQTT–SN, using the libcoap and Paho MQTT libraries, respectively. These implementations leverage wolfSSL as the basis for DTLS 1.3 and its integration with liboqs. Preliminary benchmarking for libcoap showed that the time taken for a GET request is comparable to the DTLS handshake time. Our results demonstrated that the Kyber512 outperforms the traditional P–256 algorithm in this scenario. For MQTT–SN, the subscription time was an order of magnitude higher than the handshake time, indicating a negligible impact of introducing PQC on the overall performance of certain protocol operations. Our testing on a Raspberry Pi demonstrates the feasibility of employing PQC algorithms on constrained devices.

Our experience underscores the importance of ongoing PQC integration into (D)TLS libraries. As this integration matures, the migration to PQC will become more seamless for protocol libraries. It is crucial for protocol libraries reliant on (D)TLS to closely monitor and support PQ–enabled libraries tailored to their specific use cases, whether embedded or otherwise. This will ensure a smooth transition to post–quantum secure communication and crypto–agility in IoT systems.

Our goal is to evaluate the performance and practicality of these integrations on highly resource–constrained hardware platforms, such as microcontrollers. Specifically, we plan to assess our implementations on devices like the ESP32, Milk–V Duo, and Arduino Nano RP2040 Connect. By focusing on the integration of PQC in IoT protocols and testing their performance on severely constrained devices, we aim to contribute to the development of secure IoT systems that can withstand the threats posed by quantum computers.

Acknowledgments

This work was supported under the grant TED–2021–130369B–C32 funded by MICIU/AEI/10.13039/501100011033 and by the "European Union NextGenerationEU/PRTR" and the grant PID2020–113795RB–C32 funded by MICIU/AEI/10.13039/501100011033. In addition, it was partially supported by project i-SHAPER, which is being carried out within the framework of the Recovery, Transformation, and Resilience Plan funds, funded by the European Union (Next Generation).

References

[1] Peter W Shor. Algorithms for quantum computation: discrete logarithms and factoring. In *Proceedings 35th annual symposium on foundations of computer science*, pages 124–134. Ieee, 1994.

[2] Lov K Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, pages 212–219, 1996.

[3] M. Mosca and M. Piani. Global Risk Institute: Quantum Threat Timeline Report 2022. https://globalriskinstitute.org/publication/2022-quantum-threat-timeline-report/, 2022. Accessed: 6 march 2024.

[4] Daniel J Bernstein and Tanja Lange. Post-quantum cryptography. *Nature*, 549(7671):188–194, 2017.

[5] National Institute of Standards and Technology. Draft fips (federal information processing standards) 203, module-lattice-based key-encapsulation mechanism standard. Technical report, NIST, Information Technology Laboratory, 2003.

[6] National Institute of Standards and Technology. Draft fips (federal information processing standards) 204, module-lattice-based digital signature standard. Technical report, NIST, Information Technology Laboratory, 2003.

[7] National Institute of Standards and Technology. Draft fips (federal information processing standards) 205, stateless hash-based digital signature standard. Technical report, NIST, Information Technology Laboratory, 2003.

[8] NSA CISA and NIST. Quantum-readiness: Migration to post-quantum cryptography. https://media.defense.gov/2023/Aug/21/2003284212/-1/-1/0/CSI-QUANTUM-READINESS.PDF, Aug 2023. Accessed: 15 february 2024.

[9] National Cyber Security Centre (NCSC). Quantum security technologies. https://www.ncsc.gov.uk/pdfs/whitepaper/quantum-security-technologies.pdf, Mar 2020. Accessed: 6 march 2024.

[10] National Security Agency (NSA). Quantum key distribution (qkd) and quantum cryptography (qc). https://www.nsa.gov/Cybersecurity/Quantum-Key-Distribution-QKD-and-Quantum-Cryptography-QC/. Accessed: 2024-02-15.

[11] Centro Criptológico Nacional (CCN)-PYTEC. Recomendaciones para una transición postcuántica segura, December 2022. CCN-TEC 009.

[12] French Cybersecurity Agency (ANSSI), Federal Office for Information Security (BSI), Netherlands National Communications Security Agency (NLNCSA), and Swedish Armed Forces Swedish National Communications Security Authority. Position paper on quantum key distribution. Technical report, January 2024.

[13] Douglas Stebila, Scott Fluhrer, and Shay Gueron. Hybrid key exchange in tls 1.3. Technical report, Internet-Draft, Internet Engineering Task Force (IETF) Network Working Group, Sep 2023. Draft-Ietf-Tls-Hybrid-Design-09.

[14] S. Fluhrer, P. Kampanakis, D. McGrew, and V. Smyslov. Mixing preshared keys in the internet key exchange protocol version 2 (ikev2) for post-quantum security. Technical report, Internet Engineering Task Force (IETF), RFC 8784, 6 2020.

[15] Seungyeon Bae, Yousung Chang, Hyeongjin Park, Minseo Kim, and Youngjoo Shin. A performance evaluation of ipsec with post-quantum cryptography. In Seung-Hyun Seo and Hwajeong Seo, editors, *Information Security and Cryptology – ICISC 2022*, pages 249–266, Cham, 2023. Springer Nature Switzerland.

[16] A.M. Fregly, R. van Rijswijk-Deij, M. Müller, P. Thomassen, C. Schutijser, and T. Chung. Research agenda for a post-quantum dnssec. Technical report, IEFT Network Working Group, 9 2023.

[17] Tiago M Fernández-Caramés. From pre-quantum to post-quantum iot security: A survey on quantum-resistant cryptosystems for the internet of things. *IEEE Internet of Things Journal*, 7(7):6457–6480, 2019.

[18] Kevin Bürstinghaus-Steinbach, Christoph Krauß, Ruben Niederhagen, and Michael Schneider. Post-quantum tls on embedded systems: Integrating and evaluating kyber and sphincs+ with mbed tls. In *Proceedings of the 15th ACM Asia Conference on Computer and Communications Security*, pages 841–852, 2020.

[19] Zach Shelby, Klaus Hartke, and Carsten Bormann. The Constrained Application Protocol (CoAP). RFC 7252, June 2014.

[20] Andy Stanford-Clark and Hong Linh Truong. MQTT for sensor networks (mqtt-sn) protocol specification. *International business machines (IBM) Corporation version*, 1(2):1–28, 2013.

[21] Olaf Bergmann and other contributors. libcoap: A coap (rfc 7252) implementation in c. https://github.com/obgm/libcoap, 2024. Available online: https://github.com/obgm/libcoap.

[22] Eclipse Paho Project. Paho c mqtt-sn gateway and libraries for embedded systems. https://github.com/eclipse/paho.mqtt-sn.embedded-c, 2024. Accessed: 2024-02-15.

[23] libcoap Contributors. *libcoap 4.3.4 Documentation: coap-server*. libcoap.net, 2023. Available online: https://libcoap.net/doc/reference/4.3.4/man_coap-server.html.

[24] Catarina Silva, Vitor A Cunha, João P Barraca, and Rui L Aguiar. Analysis of the cryptographic algorithms in iot communications. *Information Systems Frontiers*, pages 1–18, 2023.

[25] Victor Seoane, Carlos Garcia-Rubio, Florina Almenares, and Celeste Campo. Performance evaluation of coap and mqtt with security support for iot environments. *Computer Networks*, 197:108338, 2021.

[26] Hyeokjin Kwon, Jiye Park, and Namhi Kang. Challenges in deploying coap over dtls in resource constrained environments. In *Information Security Applications: 16th International Workshop, WISA 2015, Jeju Island, Korea, August 20–22, 2015, Revised Selected Papers 16*, pages 269–280. Springer, 2016.

[27] Douglas Stebila and Michele Mosca. Post-quantum key exchange for the internet and the open quantum safe project. In *Selected Areas in Cryptography (SAC) 2016*, volume 10532 of *Lecture Notes in Computer Science*, pages 1–24. Springer, oct 2017.

[28] Open Quantum Safe Project. liboqs: Library for quantum-resistant cryptographic algorithms. https://github.com/open-quantum-safe/liboqs, 2024. Accessed: 2024-03-10.

[29] Open Quantum Safe. OQS Provider - OpenSSL 3 provider containing post-quantum algorithms. https://github.com/open-quantum-safe/oqs-provider.

[30] Clean, portable, tested implementations of post-quantum cryptography. https://github.com/PQClean/PQClean, 2018.

[31] wolfSSL. DTLS 1.3 support for Post-Quantum Cryptography. https://www.wolfssl.com/dtls-1-3-support-post-quantum-cryptography/, January 2023. Do you want to start using wolfSSL's DTLS 1.3 implementation?

[32] Post-quantum crypto library for the arm cortex-m4. https://github.com/mupq/pqm4, 2019.

[33] Oscar M Guillen, Thomas Pöppelmann, Jose M Bermudo Mera, Elena Fuentes Bongenaar, Georg Sigl, and Johanna Sepulveda. Towards post-quantum security for iot endpoints with ntru. In *Design, Automation & Test in Europe Conference & Exhibition (DATE), 2017*, pages 698–703. IEEE, 2017.

[34] Johanna Sepúlveda, Shiyang Liu, and Jose M Bermudo Mera. Post-quantum enabled cyber physical systems. *IEEE Embedded Systems Letters*, 11(4):106–110, 2019.

[35] Gorjan Alagic, Daniel Apon, David Cooper, Quynh Dang, Thinh Dang, John Kelsey, Jacob Lichtinger, Carl Miller, Dustin Moody, Rene Peralta, et al. Status report on the third round of the nist post-quantum cryptography standardization process. *US Department of Commerce, NIST*, 2022.

[36] Ruben Gonzalez, Andreas Hülsing, Matthias J Kannwischer, Juliane Krämer, Tanja Lange, Marc Stöttinger, Elisabeth Waitz, Thom Wiggers, and Bo-Yin Yang. Verifying post-quantum signatures in 8 kb of ram. In *Post-Quantum Cryptography: 12th International Workshop, PQCrypto 2021, Daejeon, South Korea, July 20–22, 2021, Proceedings 12*, pages 215–233. Springer, 2021.

[37] Ruben Gonzalez and Thom Wiggers. Kemtls vs. post-quantum tls: Performance on embedded systems. In *International Conference on Security, Privacy, and Applied Cryptography Engineering*, pages 99–117. Springer, 2022.

[38] George Tasopoulos, Jinhui Li, Apostolos P Fournaris, Raymond K Zhao, Amin Sakzad, and Ron Steinfeld. Performance evaluation of post-quantum tls 1.3 on resource-constrained embedded systems. In *International Conference on Information Security Practice and Experience*, pages 432–451. Springer, 2022.

[39] George Tasopoulos, Charis Dimopoulos, Apostolos P Fournaris, Raymond K Zhao, Amin Sakzad, and Ron Steinfeld. Energy consumption evaluation of post-quantum tls 1.3 for resource-constrained embedded devices. *Cryptology ePrint Archive*, 2023.

[40] Jose-Antonio Septien-Hernandez, Magali Arellano-Vazquez, Marco Antonio Contreras-Cruz, and Juan-Pablo Ramirez-Paredes. A comparative study of post-quantum cryptosystems for internet-of-things applications. *Sensors*, 22(2):489, 2022.

[41] Callum McLoughlin, Clémentine Gritti, and Juliet Samandari. Full post-quantum datagram tls handshake in the internet of things. In *International Conference on Codes, Cryptology, and Information Security*, pages 57–76. Springer, 2023.

[42] wolfSSL Inc. *wolfSSL Documentation: OpenSSL Compatibility*, 2023. Copyright ©2023 wolfSSL Inc.

[43] Javier Blanco-Romero. qursa-uc3m/libcoap-wolfssl. wolfssl integration in libcoap enabling dtls 1.3 and post-quantum crypto. https://github.com/qursa-uc3m/libcoap-wolfssl, 2024.

[44] Jon Shallow. wolfSSL: Set up initial port #1358, 2024. Pull request in the libcoap repository.

[45] Javier Blanco-Romero. qursa-uc3m/paho.mqtt-sn.embedded-c.wolfssl-pq. wolfssl integration into paho c mqtt-sn gateway and libraries for embedded systems for post-quantum cryptography evaluations. https://github.com/qursa-uc3m/paho.mqtt-sn.embedded-c.wolfssl-pq, 2024.